

BÀI THỰC HÀNH: THUẬT TOÁN TÌM KIẾM VÀ SẮP XẾP

1. MỤC TIÊU BÀI THỰC HÀNH

Sau khi hoàn thành bài thực hành này, sinh viên có khả năng:

- Hiểu và cài đặt các **phương pháp tìm kiếm cơ bản**: tìm kiếm tuần tự, tìm kiếm nhị phân, tìm kiếm bằng bảng băm.
- Hiểu và cài đặt các **thuật toán sắp xếp hiệu quả**: Quick Sort và Merge Sort.
- So sánh đặc điểm, độ phức tạp và phạm vi ứng dụng của từng thuật toán.
- Cài đặt chương trình bằng **ngôn ngữ Java**, tuân thủ cấu trúc chương trình chuẩn.

2. CÁC VÍ DỤ MINH HỌA (CHƯƠNG TRÌNH HOÀN CHỈNH)

Ví dụ 1: Tìm kiếm tuần tự (Linear Search)

```
public class LinearSearch {  
    public static int linearSearch(int[] a, int x) {  
        for (int i = 0; i < a.length; i++) {  
            if (a[i] == x) return i;  
        }  
        return -1;  
    }  
    public static void main(String[] args) {  
        int[] a = {5, 2, 9, 1, 7};  
        int x = 9;  
        int pos = linearSearch(a, x);  
        System.out.println(pos == -1 ? "Không tìm thấy" : "Tìm thấy tại vị trí " + pos);  
    }  
}
```

Ví dụ 2: Tìm kiếm nhị phân (Binary Search)

```
import java.util.Arrays;

public class BinarySearch {
    public static int binarySearch(int[] a, int x) {
        int left = 0, right = a.length - 1;
        while (left <= right) {
            int mid = (left + right) / 2;
            if (a[mid] == x) return mid;
            if (a[mid] < x) left = mid + 1;
            else right = mid - 1;
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a = {5, 2, 9, 1, 7};
        Arrays.sort(a);
        int x = 7;
        int pos = binarySearch(a, x);
        System.out.println(pos == -1 ? "Không tìm thấy" : "Tìm thấy tại vị trí " + pos);
    }
}
```

Ví dụ 3: Tìm kiếm bằng bảng băm (Hash Table)

```
import java.util.HashMap;

public class HashSearch {
    public static void main(String[] args) {
        int[] a = {10, 22, 5, 17, 8};
```

```
HashMap<Integer, Integer> map = new HashMap<>();

for (int i = 0; i < a.length; i++) {
    map.put(a[i], i);
}

int x = 17;
if (map.containsKey(x)) {
    System.out.println("Tim thay " + x + " tai vi tri " + map.get(x));
} else {
    System.out.println("Khong tim thay");
}
}
}
```

Ví dụ 4: Thuật toán Quick Sort

```
public class QuickSort {
    static void quickSort(int[] a, int low, int high) {
        if (low < high) {
            int p = partition(a, low, high);
            quickSort(a, low, p - 1);
            quickSort(a, p + 1, high);
        }
    }

    static int partition(int[] a, int low, int high) {
        int pivot = a[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (a[j] <= pivot) {
                i++;
                int temp = a[i]; a[i] = a[j]; a[j] = temp;
            }
        }
    }
}
```

```

    }
    int temp = a[i + 1]; a[i + 1] = a[high]; a[high] = temp;
    return i + 1;
}

public static void main(String[] args) {
    int[] a = {9, 4, 7, 3, 10, 5};
    quickSort(a, 0, a.length - 1);
    for (int x : a) System.out.print(x + " ");
}
}

```

Ví dụ 5: Thuật toán Merge Sort

```

public class MergeSort {
    static void mergeSort(int[] a, int left, int right) {
        if (left < right) {
            int mid = (left + right) / 2;
            mergeSort(a, left, mid);
            mergeSort(a, mid + 1, right);
            merge(a, left, mid, right);
        }
    }

    static void merge(int[] a, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;
        int[] L = new int[n1];
        int[] R = new int[n2];
        for (int i = 0; i < n1; i++) L[i] = a[left + i];
        for (int j = 0; j < n2; j++) R[j] = a[mid + 1 + j];

        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {

```

```

if (L[i] <= R[j]) a[k++] = L[i++];
else a[k++] = R[j++];
}
while (i < n1) a[k++] = L[i++];
while (j < n2) a[k++] = R[j++];
}
public static void main(String[] args) {
int[] a = {8, 3, 2, 9, 7, 1, 5};
mergeSort(a, 0, a.length - 1);
for (int x : a) System.out.print(x + " ");
}
}

```

Ví dụ 6: Sắp xếp sử dụng Arrays.sort() (Thư viện chuẩn Java)

```

import java.util.Arrays;
import java.util.Collections;
public class ArraysSortDescending {
    public static void main(String[] args) {
        // Mảng đối tượng Integer
        Integer[] a = {12, 4, 7, 1, 9, 3};
        // Sắp xếp giảm dần
        Arrays.sort(a, Collections.reverseOrder());
        System.out.println("Mang sau khi sap xep giam dan:");
        for (Integer x : a) {
            System.out.print(x + " ");
        }
    }
}

```

3. NỘI DUNG BÀI THỰC HÀNH

Bài 1: Tìm kiếm sản phẩm trong hệ thống bán hàng

Bối cảnh thực tế: Một cửa hàng online cần tìm nhanh sản phẩm theo mã sản phẩm.

Yêu cầu:

- Sinh danh sách sản phẩm (mã SP, tên, giá).
- Cho phép người dùng nhập mã sản phẩm cần tìm.
- Áp dụng tìm kiếm nhị phân (Sắp xếp trước khi tìm bằng QuickSort hoặc Merge Sort) và bảng băm.
- Nhận xét phương pháp phù hợp nhất trong thực tế.

Bài 2: Đánh giá hiệu năng thuật toán

Bối cảnh thực tế: Kỹ sư phần mềm cần chọn thuật toán phù hợp cho dữ liệu lớn.

Yêu cầu:

- Sinh mảng dữ liệu lớn ($N = 10^5$ trở lên).
- Đo thời gian tìm kiếm và sắp xếp.